A Longitudinal Study of Android Apps Signing Key Protection

Mark Huasong Meng
University College Dublin
Dublin, Ireland
mark.meng@ucd.ie

Qing Zhang
JD Group
Beijing, China
zhangqingsdu@gmail.com

Weirao Lu
JD Group
Beijing, China
wrlu.cn@outlook.com

Chunyang Chen✉
TU Munich
Heilbronn, Germany
chun-yang.chen@tum.de

Abstract

Android app signing relies on developer-managed credentials, making secure key protection essential for the integrity of the software supply chain. A recent platform key leakage incident involving two major OEM manufacturers demonstrates that even robustly designed signing mechanisms can be compromised due to developers' oversight. In this work, we conduct a longitudinal ecosystem study to characterize this threat by mining public repositories for Android signing credentials, recovering compromised keys via exposed passwords, and matching them against signatures from over 4,000 apps collected from major stores and OEM system images. Our analysis identifies 5,673 compromised keystores on GitHub and 26 unique certificates linked to 278 real-world apps. These include 26 third-party apps in public app stores and 252 preinstalled apps from seven manufacturers, collectively affecting over 10 billion users. We demonstrate the practical exploitability of these leaks through a proof-of-concept app replacement attack and identify spillover risks in non-smartphone platforms, including a popular automotive head-unit platform installed in over 1,100 vehicle models. Our results reveal that signing-key mismanagement is a systemic risk, underscoring the need for more rigorous key-management support in Android release engineering and distribution infrastructures.

CCS Concepts

• **Security and privacy → Software security engineering**; • **Software and its engineering → Software post-development issues**.

Keywords

Security, Mobile Apps, Mining Software Repositories, Software Supply Chain.

ACM Reference Format:
Mark Huasong Meng, Qing Zhang, Weirao Lu, and Chunyang Chen. 2026. A Longitudinal Study of Android Apps Signing Key Protection. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3832783.3834428

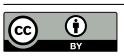
1 Introduction

App signing serves as the primary security mechanism to protect the integrity and authenticity of applications (apps) after they are released by developers. As in most operating systems (OSes), Android is designed with an app signing framework that forms a critical component of its security architecture [11]. When developers release their apps, Android requires all apps to be digitally signed with the developer's cryptographic keys before distribution. During installation, the Android OS verifies the app's signature, ensuring that only apps signed by the same developer can replace or upgrade existing versions. This mechanism not only safeguards app integrity but also enables advanced features such as permission-based privilege sharing between apps released by the same developer.

In recent years, critical vulnerabilities like Log4Shell have pushed software supply chain security to the forefront of cybersecurity concerns [23, 30, 31]. As app signing constitutes a critical component within the software supply chain, its reliability directly impacts the security of the entire Android ecosystem. Unfortunately, the Android app signing mechanism has indeed exposed several significant vulnerabilities in the past [40]. Notable vulnerabilities, such as the Master Key vulnerability (CVE-2013-4787) [28] and the Janus vulnerability (CVE-2017-13156) [29], have exposed fundamental design flaws in the signing mechanism. These issues were not comprehensively addressed until the release of Android 9, after which the app signing mechanism has not exhibited severe flaws, with remaining risks largely attributed to whether developers follow secure development practices and adequately protect their signing keys.

In this context, the security implications of signing-key compromise are not limited to apps distributed through Google Play. In practice, many Android apps are obtained outside Google Play's security controls, including apps released in alternative regional markets, apps on devices without Google Play services, and OEM-preinstalled apps that are not listed in the Play Store. In these settings, Google Play's store-side protections [20] do not uniformly apply, making developer- and OEM-side signing-key protection particularly important and leaving greater room for the consequences of signing-key mismanagement to surface in real-world deployments.

In this work, we investigate the security landscape of Android app signing with a particular focus on the risk of signature leakage. Our research is motivated by a recent OEM platform key leakage incident [27], which prompted us to examine whether developers' mismanagement of signing keys could pose a broader threat beyond individual apps and across the broader Android ecosystem, including smart home platforms and on-vehicle head units. Moreover, our concerns are elevated by findings in recent literature [40] indicating that some amateur Android developers use publicly known keys to sign their applications. Once such keys are leaked, attackers can craft malicious apps bearing authentic signatures. Given Android's unique openness and its substantial market share, there exists a strong demand for a comprehensive



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3834428

assessment to strengthen confidence in the security of the Android software supply chain.

To address these concerns, we conduct a large-scale empirical study of the Android app signing mechanism. Specifically, we longitudinally crawl public repositories to collect signing credentials and identify compromised keys by searching for associated passwords within the same repositories. In parallel, we gather over 4,000 apps from major app stores and OEM system images. By matching app signatures with the compromised signing keys, we evaluate the potential impact of signing key leakage issues and reveal the security landscape of the app signing mechanism in the Android ecosystem. Finally, we design two manipulation strategies and implement a proof-of-concept attack on real devices to demonstrate the exploitability of our findings. We also discuss the broader risks posed by signing key leakage, extending beyond smartphone devices to other Android-based platforms. Among many interesting results and observations, the following are the most prominent:

- **Mismanagement of app signing keys is prevalent in the Android ecosystem**. 5,673 unique keys are found compromised on public repositories on GitHub.
- **26 apps released on mainstream app stores are found to be vulnerable** due to the compromised signing keys. These apps collectively account for over 140 billion billion reported installs across seven app stores.
- **252 preinstalled apps are signed with 13 compromised keys on devices from 7 manufacturers**, posing severe threats to the users' devices if a malicious replacement app misuses privileged permissions or launches a Denial-of-Service (DoS) attack.
- **A proof-of-concept attack** that can be installed to replace a benign real-world app on our tested device, demonstrating the exploitability of the signing key leakage issues.
- **A signing key used by a third-party automotive head-unit platform was found to be compromised**. The platform has been installed in over 1,100 vehicle models since 2019, indicating that the risks extend beyond smartphones to other Android-based platforms.

To the best of our knowledge, this is the first systematic study of potential leakage of app signing keys in the Android ecosystem at scale and over time. Our findings should alert both developers and OEM manufacturers to comply with rigorous security practices throughout the entire development life cycle and urge the industry to adopt more secure key management solutions.

2 Background

2.1 Android App Signing & Trust Relationships

Publishing an Android app is not only about transforming functionality requirements into source code; developers are also expected to protect their apps' integrity and authenticity from spoofing attacks. In the Android ecosystem, app signatures are used to verify package integrity and authenticate the developer's signing identity [11].

The process of app signing is depicted in Figure 1. Specifically, a developer needs to generate a key pair consisting of a private key and a public key, create a certificate that specifies developer identity information, and sign the app package with the private key and certificate. The key pair is usually saved in Java keystore

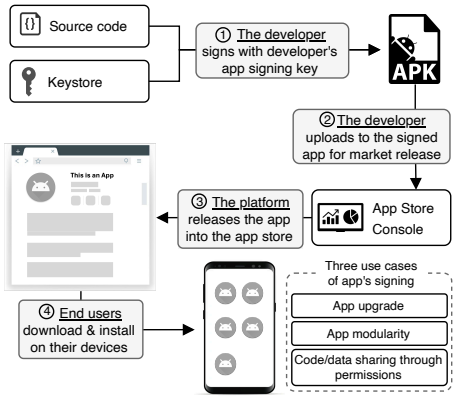


Figure 1: The role of app signing in the life cycle of an Android app

files (with extensions `.jks` or `.keystore`). When developers finish the implementation of an app, they use the private key to generate a signature over the installation package (an `.apk` file, hereafter referred to as an APK). The resulting signature and signing certificate, but not the private key, are included in the APK during the release process (step 1) [12]. In practice, developers can resort to development tools (e.g., Android Studio) to generate keys and sign an app through a graphical user interface (GUI) with a few clicks. Corporate developers who simultaneously develop and maintain multiple apps can also configure the signing process into the development pipeline, for example, by specifying the signing key and password in a "`Build.gradle`" file in the project.

After the signing, the developer uploads the signed package to the app store console provided by the release platform, e.g., Google Play App Store (step 2). The release platform will publish the app after performing integrity verification (step 3) and then make it public for end users to download and install on their devices (step 4). The OS on the users' devices checks the signature during the installation, and only allows app upgrades if both the new and old versions of an app are signed by the same developer. Signature continuity can also mediate certain trust relationships between apps signed by the same developer, including signature-level permission sharing [17]. Consequently, leakage of a signing key can enable an attacker to distribute forged apps that inherit the developer's signing identity.

2.2 Threat Model and Scope

In this work, we study *developer-side exposure* of Android app signing credentials in public code repositories. Our focus is not on cryptographic flaws in Android signature schemes, but on operational key-management failures that expose keystore files and associated passwords during development and release engineering. This threat is significant because Android relies on signature continuity to authenticate app updates and to support same-signer trust relationships. In practice, signing keys are long-lived and difficult

to replace without disrupting an app's normal lifecycle. Google recommends that private signing keys should remain valid for at least 25 years [11]. App developers are also responsible for securing their keys and the containing keystore files. Google also suggests that developers maintain the keystore files in hardware or services that require multiple factors of authentication to maximize signing-key protection [13].

We consider an attacker who can access artifacts unintentionally exposed in a public repository, including keystore files and related build configuration files. The attacker is assumed to inspect the same repository for information that enables keystore recovery, such as plaintext passwords or file references to password material. Under this threat model, if a signing key can be recovered, the attacker may produce a forged app package (i.e., an `.apk` file) bearing the victim developer's signing identity. This forged package could then be used to attempt a malicious update, impersonate a same-signer app, or abuse signature-based trust relationships. The exact impact depends on where the affected app is deployed and what privileges it holds.

Our analysis covers two major app deployment contexts. The first is third-party apps distributed through mainstream app stores. The second is preinstalled or system apps on OEM devices[1], where compromise may be especially serious because signature-based trust can interact with privileged permissions. We also discuss broader implications for other Android-based platforms when relevant keys are reused outside the smartphone ecosystem.

It is worth noting that our analysis scope excludes cases where a keystore is present online but cannot be linked to a real-world app, as well as cases where the signing identity is observed in an app but the corresponding keystore cannot be recovered (i.e., password not found). For that reason, our study separates *potential leakage* from *confirmed compromise*. Specifically, matching a repository-exposed signing identity (i.e., certificate fingerprint) to a released app indicates potential exposure, whereas successful recovery and use of the signing key demonstrates practical exploitability.

3 Study Methodology

We present the details of our study in this section. We first introduce the study design and define four research questions that we focus on in this paper. We then outline our methodology as a three-stage pipeline and briefly introduce different steps. Finally, we present the detailed methodology.

3.1 Study Design

In this paper, we focus on the following research questions:

- **RQ1 Signing Key Mismanagement.** How many Android app signing keys are hosted on public repositories on GitHub? Can we find any keys compromised due to password leakage?
- **RQ2 App Signature Leakage.** Have the compromised keys, if any, been used for signing real-world apps on Android devices? How many apps are vulnerable due to the mismanagement of app signing keys?
- **RQ3 Potential Impact.** What is the potential impact of signing-key compromise? How many users or devices may be affected?
- **RQ4 Exploitation.** Can compromised signing keys be exploited in practice to threaten affected users or devices?

[1]We refer to "system apps" as apps installed in system directories (e.g., "`/system/app/`", "`/system/priv-app/`", etc.) and therefore might be given certain privileged permissions.

3.2 Approach Overview

Our study follows a three-stage pipeline, as illustrated in Figure 2. In **Stage 1** (*repository-side compromise identification*), we first crawl keystore files for Android app signing from public GitHub repositories (step 1) and then attempt to recover their signing keys by inspecting the same repositories for exposed passwords and verifying keystore access (step 2). We present more details of this stage in Section 3.3.

In **Stage 2** (*real-world app matching*), we collect signatures from released Android apps, including both third-party apps from mainstream app stores and preinstalled system apps from OEM devices (step 3), and match them against the certificate fingerprints of the recovered signing keys (step 4). A match indicates that a compromised signing key has been used by a real-world app. We detail stage 2 in Section 4. The security implications of these matched cases are analyzed later in Section 5.3.

Finally, in **Stage 3** (*exploitability validation*), we use the recovered keys to sign forged apps and conduct a spoofing attack. Specifically, we assess whether the forged apps can be accepted as legitimate updates (step 5). A successful spoofing attack demonstrates the practical exploitability of a previously recovered signing key. All apps signed by that key are exposed to malicious manipulation. We detail this stage in Section 6.

With the above three stages, our pipeline connects repository-side evidence to real-world app exposure and practical exploit validation.

3.3 Repository-Side Compromise Identification

Crawling Keystores from Public Repositories. Leakage of an app-signing key may arise from multiple sources, including unauthorized theft or inadvertent public exposure of the corresponding keystore file. In this work, we focus on the latter. Collecting keystores obtained through other attack channels at scale would be both technically difficult and ethically inappropriate, whereas publicly exposed repository artifacts provide a measurable and reproducible source of developer-side signing-key mishandling.

We therefore target public GitHub repositories, which constitute a large platform for hosting Android source code. Because exhaustive extension-based retrieval is constrained by GitHub search limitations, we design a three-phase collection approach using RESTful API requests:

(1) **Identifying the repositories**. Since our goal is to collect keystores used for Android app signing, we first identify repositories that are likely to host Android projects. We require candidate repositories to contain a build configuration file with the "`.gradle`" extension [14]. To further improve precision, we search for signing-related keywords derived from the `signingConfigs` section of a typical "`Build.gradle`" file, including "keyAlias", "storeFile", "storePassword", and "keyPassword" [11]. To avoid missing repositories due to GitHub result limits, we partition the search space by constraining the size of the target `.gradle` files and iterating over file size ranges from zero KB to 100 MB.

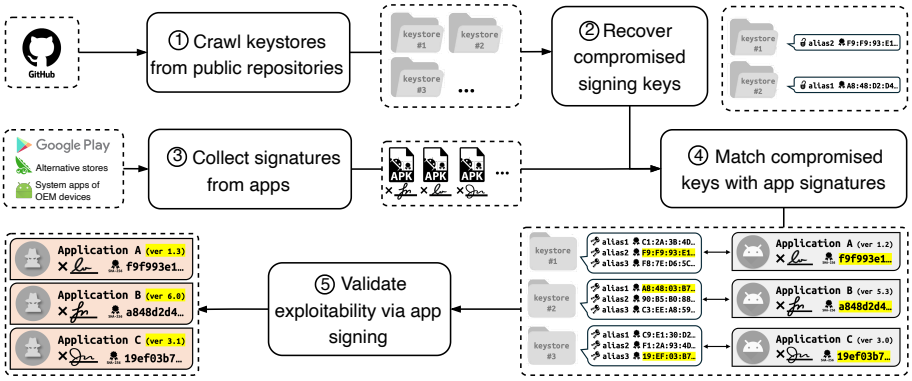


Figure 2: Overview of our methodology, including repository-side compromise identification, ecosystem-side signature matching, and exploit deployment

(2) **Collecting keystores from repositories**. For each identified repository, we download the repository archive as a ZIP file[2] and inspect its file list for keystore artifacts with the "`.jks`" or "`.keystore`" extension. Whenever such a file is found, we add it to our candidate keystore set for further validation.

(3) **Validating the keystores**. Not all files with these extensions are valid keystores. Some are placeholders, such as files containing strings like "redacted" or "replace me with your keystore". We therefore validate each collected file using Java `keytool` and retain only those that contain at least one private-key/certificate pair suitable for Android app signing.

Because our goal is to characterize the evolving landscape of Android signing-key exposure, we perform this collection longitudinally rather than as a one-time crawl. Our keystore collection begins in April 2023 and is repeated monthly until August 2024.

Recovering Compromised Signing Keys. Releasing the keystore files on a public repository does not directly imply a security breach unless there is a way to efficiently access the private keys embedded in these keystores. While our work does not aim to crack these keystores, we instead resort to a simple but straightforward approach to search for the password in plaintext within the same repository. This is motivated by occasional reports in the past that some developers hardcoded the passwords to access their keystore files and private keys in the configuration file, i.e., "`Build.gradle`" [22]. We regard the key signing process in the Android ecosystem as centrally managed in the configuration files, and accordingly search for them in the same repository that stores the keystore files. Specifically, we revisit these configuration files, parse the passwords in their signing configuration section (i.e., the `storePassword` and `keyPassword` elements in `signingConfigs`), and treat the keystores as *recoverable* if either of the following conditions holds: (1) the password is hardcoded in plaintext, or (2) its value references a publicly accessible file in the same repository.

Next, we decrypt the keystore files using the passwords crawled from the same repositories. The correctness of a keystore password can be verified using `keytool`. In this work, we use the `keytool -list` command to verify candidate keystore passwords. If the supplied password is correct, the command successfully lists the entries in the keystore, including their aliases and certificates.

4 Identifying App Signature Leakage

Collecting Signatures from Apps. In this step, we aim to collect APKs from a broad range of applications to maximize signature coverage. To this end, our app collection starts with downloading the top 1,000 non-game free apps as of April 2023 from the Google Play App Store, given that it is the official source of third-party apps in the Android ecosystem. Due to some restrictions imposed by Google, we cannot directly download APK files to non-Android devices from its app store. Instead, we take the app ranking from the Google Play App Store, retrieve the package names of these apps, and download them through a third-party app repository named "APKPure".[3] We also consider that the absence of Google Play services in China might limit the scope and impact assessment of our investigation, and therefore include "Wandoujia"[4], a third-party app market with a large user base in China, as an alternative source of APK files to complement our app collection. In the same way, we collect the identifiers of another 1,000 non-game free apps[5] from the top list of the alternative app stores and download the APK files from APKPure to support lightweight retrieval of current APK versions.

In addition to third-party apps circulating in the mainstream app stores, we also collect the preinstalled apps on different OEM devices. These apps include preinstalled third-party apps that may be granted privileged permissions by the OEM manufacturers (also referred to as OEM vendors in the relevant study) and the system apps signed by the OEM manufacturers themselves, because leakage of a manufacturer key could pose broader ecosystem risk. Specifically, we dump the preinstalled apps and the system apps[6] from

[2]The zip file can be directly downloaded through the formatted URL https://github.com/<username>/<repository-name>/archive/<branch-name>.zip.

[3]Available at https://apkpure.com/app

[4]Available at https://www.wandoujia.com/

[5]Our collection does not contain duplicates in the two app stores. Thus, when we collect apps from Wandoujia, we skip the app if it has already been downloaded in the previous collection on the Google Play Store.

[6]APKs located in `/system/framework/` directory, e.g., `framework-res.apk`.



(a) Keystore passwords (b) Signing key passwords

Figure 3: Word cloud pictures showing the distributions of passwords leaked on public repositories (only the 40 most frequently used passwords are shown due to space limit)

11 devices, representing different hardware models produced by Google Pixel and third-party OEM manufacturers including Honor, Huawei, Infinix, Meizu, OnePlus, Oppo, Samsung, Tecno, Vivo, and Xiaomi. Together, these manufacturers cover 87.5% of the market share in the Android smartphone and tablet ecosystem at the time of the study being performed [2].

For each APK file downloaded, we resort to a built-in tool of Android SDK named "apksigner" to retrieve the app signature. Specifically, the certificate digest, signer's details, and public key digest of each app are collected for the next step of investigation.

Matching Compromised Keys with App Signatures. After collecting keystores and app signatures from the Internet, we then explore whether any certificates match between the two sets. Specifically, we extract all certificate digests from the APK files, and compare each of them with the *certificate fingerprints* recovered from the collected keystore binaries. We depict this process in step 4 of Figure 2. If an APK's signing certificate has the same digest as a certificate recovered from a collected keystore, we determine that the corresponding key was used to sign the app, and the corresponding keystore contains the key matching the app's signing certificate. In the same way, we regard this as a case of signing key leakage for the victim app and its developer(s).

Unlike signing keys leaked on the Internet, certificates extracted from APK files usually contain lots of metadata about the developers, including countries, company and department names, and email addresses. Attackers can use this information to infer the market share and impact of a compromised signing key once matched with any app's signatures.

5 Results and Analysis

5.1 RQ1: Signing Key Mismanagement

Through our large-scale longitudinal collection, we managed to collect 56,510 keystore files from public repositories on GitHub. Among these 56,510 keystores, we find 11,526 unique keys (i.e., private key entries) by their certificate fingerprints (in the format of SHA-256 digests).

Among the 56,510 keystores retrieved from online repositories, 46,619 (82.5%) keystores are uploaded together with passwords in plaintext. We further remove duplicates by counting the unique fingerprint of the keystore files and recognizing 5,673 unique compromised keystores, in which 6,602 signing keys are identified.

We also observe common patterns in the leaked passwords. Specifically, we find that "`android`", "`password`", and "`123456`" are the top three widely used passwords in both compromised keystores and signing keys. We illustrate the distribution of the leaked passwords in Figure 3. Our statistics also show that the vast majority of compromised signing keys (5,379 out of 6,602, 81.5%) use the same password as their containing keystores.

5.2 RQ2: Signature Leakage of Real-World Apps

Our app collection managed to identify 4,196 unique apps involving 2,225 distinct developer signatures. 259 are found to be signed by multiple certificates, ranging from 2 to 11. Among the 259 multiply signed apps, 252 (97.3%) are core system apps developed by Google or hardware vendor[7], which are signed by different OEMs during production.

Our signature matching managed to find 26 compromised certificates involving 278 apps and 75 repositories. Two compromised certificates are found in more than 10 apps. Specifically, a certificate with the fingerprint prefix "`a40da80a59`" is used by 27 apps. It is widely adopted in signing the compatibility test suite apps[8] in nine OEM devices. The other one whose fingerprint begins with "`a6ef817bfd`" exists in 17 apps. Through our analysis, we believe it is owned by one of the major players in the global mobile app market named Baidu because the package names of all affected apps start with "`com.baidu.*`". We provide further analysis in Section 5.3.

In addition to the compromised certificates, we also find a matching certificate without its password leaked on the Internet, which is found in a system app named "`com.android.sdksandbox`" in a device produced by Vivo. We believe it is owned by the OEM manufacturer because the signer email "bbktel@bbktel.com" identifies Vivo's parent company. Although we do not classify this key as compromised because its password was unavailable, public exposure of the keystore still warrants remediation and further investigation.

5.3 RQ3: Impact and Risk Analysis

Based on the signature matching outcomes, we further explore the security impact of our findings.

5.3.1 ***Impact of Involved Third-Party Apps.*** We first investigate the involved third-party apps that are publicly available on app stores. As shown in Table 1, there are nine compromised keys that have been used for signing third-party apps circulating in public app stores. Each compromised key is owned by a different developer. The 26 affected apps span 10 functional categories. We then visit the product pages of these 26 apps on app stores, and then count the number of downloads or installs based on the metadata available.

In order to understand the real user volume, we resort to the statistics from as many app stores as possible. To this end, in addition to the Google Play Store and Wandoujia, we also searched the

[7]We recognize the core system apps by package name patterns, e.g., "`com.android.*`" (153 apps), "`com.qualcomm.*`" (39), "`com.google.*`" (34), "`android.*`" (8), and "`vendor.*`" (3).

[8]Identified by their package names, "`com.android.cts.ctsshim`" and "`com.android.cts.priv.ctsshim`", with signer email "android@android.com"

Table 1: List of third-party apps involved in signing key leakage

Certificate Fingerprint	Owner	App Package Name	Category	Downloads & Installs†						
				Google Play Store	Alternative Stores‡			OEM Vendor Stores‡			
					(A1)	(A2)	(A3)	(O1)	(O2)	(O3)	(O4)
a6ef817bfd	Baidu	com.baidu.searchbox	Tools	–	3.3B	410M	34.9M	26.7B	5.4B	700M	12.8B
		com.baidu.BaiduMap	Map	5M+	2.1B	160M	16.8M	9.3B	3.7B	6.1B	479M
		com.baidu.appsearch	Utility	–	1.7B	1.2B	12.5M	–	–	–	–
		com.baidu.input	Input Method	5M+	270M	100M	9.2M	700M	230M	270M	100M
		com.baidu.browser	Browser	–	170M	74.6M	–	100M	77.6M	47.6M	46.1M
		com.baidu.searchbox.lite	Tools	–	160M	99.2M	8.5M	2.1B	940M	1.5B	1.1B
		com.baidu.searchbox.tomas	Tools	–	5.1M	300K	–	200M	140M	83.7M	210M
		com.baidu.searchbox.senior	Tools	–	113K	–	43K	–	540K	674K	640K
0a350c6fbf	Kwai (Joyo Tech)	com.smile.gifmaker	Video	–	2.4B	1.6B	62.9M	29.4M	–	–	–
		com.kuaishou.nebula	Video	–	390M	540M	23.2M	15.7B	–	–	–
		com.kwai.videoeditor	Video	–	24.7M	84.4M	1.98M	1.1B	–	–	–
		com.kwai.livepartner	Video	–	12.4M	7.4M	670K	–	–	–	–
		com.kwai.thanos	Video	–	8.1M	6M	421K	25M	–	–	–
a7a4ec5bc5	VIPSHOP	com.achievo.vipshop	Shopping	–	690M	45.8M	6.7M	14.4B	–	–	–
1315d03f03	Ctrip	ctrip.android.view	Travel	1M+	380M	32.1M	92.8M	10.6B	1.4B	3.0B	210M
b031fe98a4	Xiaomi	com.xiaomi.smarthome	Tools	50M+	140M	–	1M	1B	7.1B	–	–
		com.xiaomi.wearable	Tools	10M+	542K	–	–	9M	400M	–	–
		com.xiaomi.router	Tools	5M+	–	–	–	–	100M	–	–
		com.xiaomi.mico	Tools	–	7.7M	–	361K	200M	200M	–	–
		com.wali.live	Social	–	446M	–	–	–	–	–	–
		com.xiaomi.superhexa	Tools	–	1.3K	–	–	–	–	–	–
14ceb70dfb	Go Live	com.gau.go.launcherex	Utility	100M+							
fdf6b1e025	EastMoney	com.eastmoney.android.berlin	Finance	–	24.6M		1.4M	5M	–	–	–
		com.eastmoney.android.newyork	Finance	–	80K	240K	11K	3M	–	–	–
509e03e376	Baidu	com.baidu.carlife	Tool	–	3.7M	1.5M	524K	–	–	–	–
d1e73a6359	OneMicroWorld	com.yunio.smallworld	Social	–	272K	–	5.6K	–	–	–	–

† The data is retrieved from the public meta-data from the selected app stores as of January 2025. "–" indicates the app is not available on that store.
‡ The three alternative stores are (A1) Tencent App Store, (A2) Baidu App Store, and (A3) Wandoujia. The four OEM vendor app stores are owned by (O1) Huawei, (O2) Xiaomi, (O3) Oppo, and (O4) Vivo, respectively.

package name of the 26 apps on two additional public app stores, named Tencent app store and Baidu app store. As we can access the app stores that are privately owned by OEM vendors through our tested devices, for those OEM-owned app stores that display downloads or install numbers of an app, we additionally search the package names and record the number of downloads or installs. As a result, we present a rough estimation of user scope by composing statistics of eight app stores, and sort the signing keys by the scale of impact in Table 1. As the table shows, there are at least 8 of the 26 apps that have gained over 1 billion installs. Two apps, namely "BaiduMap" and "Ctrip", have been downloaded over 10 billion times. Although these two apps mainly target Chinese users, such a large user base can still bring serious consequences once the leaked keys have been misused by attackers.

5.3.2 ***Compromised Certificates in Pre-Installed Apps.*** Apart from apps from stores, the compromised keys are also widely used for signing pre-installed apps. Pre-installed apps signed with compromised keys may have an even greater impact if they are system or privileged apps[9] because these apps may be allowed to perform privileged operations without explicit user approval[10] [16].

[9]We refer to system and privileged pre-installed apps as the apps installed in a system directory (e.g., "/system/app/" and "/system/priv-app/") and granted "preinstalled" privilege.
[10]Unlike dangerous permissions that request user consent, privileged permissions are typically granted automatically at installation time and come with system-level trust.

Table 2: List of pre-installed third-party apps involved in leakage of signing key with its fingerprint as a6ef817bfd

App Package Name	Category	Involved Vendor	Privilege Tier†	
			System	Privileged
com.baidu.input	Input Method	Xiaomi		
		Vivo		✓
		Oppo	✓	
com.baidu.map.location	Location	Samsung		✓
		Vivo		✓
com.baidu.location.fused	Location	Samsung		✓
		Honor		✓
com.baidu.swan	Utility	Honor	✓	

† System apps are pre-installed in the "system/app/" and "product/app/" directories. Privileged apps, on the other hand, are pre-installed in the "system/priv-app/" directory. Any app that is neither a system app nor a privileged app is typically installed in the "data/app/" directory.

In this work, we find 13 compromised keys from the Internet that are used for signing pre-installed apps. Among these 13 certificates, we find that one certificate is used to sign pre-installed third-party apps. Specifically, four pre-installed third-party apps are signed with that key on our tested devices from five OEM vendors. As shown in Table 2, all these four apps are developed by Baidu and are therefore signed using the same compromised key as what we found from the app stores (refer to Table 1). Four out of five

Table 3: List of public AOSP keys used to sign real-world apps

Certificate Fingerprint	App Package Name	Involved OEM Vendors
097b2bd6d4	com.android.nearby.halfsheet	Meizu
366f75e331	com.android.safetycenter.resources	Meizu, Oppo, Vivo
43ddcf4e8a	com.android.uwb.resources	Pixel, Meizu, Oppo, Vivo
6ef8a09a1c	com.android.wifi.resources	Honor, Huawei, Meizu, Oppo, Vivo
936ef21771	com.android.hotspot2.osulogin	Honor, Huawei, Meizu, Oppo, Vivo
a6ccc500ff	com.android.bluetooth	Vivo
abf21f9e2a	com.android.sdksandbox	Meizu, Vivo
c97c5176ed	com.android.connectivity.resources	Honor, Huawei, Meizu, Oppo, Vivo
ccc51124be	com.android.adservices.api	Meizu, Oppo, Vivo
dc79bd2900	com.android.wifi.dialog	Meizu, Oppo, Vivo
e1dbadce60	com.android.captiveportallogin	Meizu
	com.android.cellbroadcastreceiver.module	Meizu
	com.android.cellbroadcastservice	Meizu
	com.android.networkstack	Meizu
	com.android.networkstack.tethering	Meizu
b031fe98a4	com.xiaomi.wearable	Xiaomi
	com.xiaomi.smarthome	Xiaomi
	com.xiaomi.superhexa	Xiaomi
	com.xiaomi.mico	Xiaomi
	com.xiaomi.router	Xiaomi

OEM vendors choose to install these apps in privileged directories, i.e., "/system", "/vendor", and "/product", and consequently these apps become system apps or even privileged apps [18, 21, 36]. For example, through our reverse engineering, we found that all four apps request access to device identifiers and the list of installed apps in their permission declaration. These data are considered highly sensitive user privacy and therefore, access to them has been banned from third-party apps since Android OS version 10 [9] and 11 [10]. However, these strict rules do not apply to system and privileged apps. If replaced through a spoofing attack, these apps may retain access to sensitive data through their existing privileged permissions, resulting in a substantial privacy risk. Moreover, the exposure of privileged app certificate also brings a risk of permanent denial-of-service of the victim device, which we refer to as "creating permission conflict" and detail in Section 6.

5.3.3 ***Misuse of AOSP Certificates.*** In addition to the one third-party-owned certificate caught in pre-install apps, the remaining 12 compromised certificates are recognized as "AOSP keys" as they are released by Google in its Android Open Source Project (AOSP) as part of the "testing kit". These certificates are uploaded together with the private keys (as the pair of .x509.pem and .pk8 files) in the /product/security directory. Google has underscored that the developers and vendors should *never use them to sign any packages in publicly released images* [19]. In other words, these 13 certificates represent "exposure by design" rather than accidental leakage on the Internet. Unfortunately, we still find six vendors, including Google Pixel, that use the AOSP keys to sign their app packages (i.e., system components and vendors' in-house apps) in their OS images. We list these certificates and involved packages in Table 3.

The potential risk of misusing "public" certificates in signing the critical system components is far greater than that of signing third-party apps. Given that these system components or vendors' in-house apps are usually highly privileged with many sensitive permissions, once the attacker knows the exact certificate used, they can easily find the private key from the public AOSP projects. An attacker who wants to misuse the privileges may not even necessarily conduct a spoofing attack since an arbitrary app signed with the platform certificate can inherit the same set of permissions.

6 Exploitation (RQ4)

Leaked app-signing keys can be exploited by malicious parties. They can forge apps on behalf of the developer if they manage to recover both the passwords for the keystore and the corresponding private key. Such forged apps may satisfy Android's package-signature checks and, under suitable deployment conditions, be accepted as updates to legitimately installed apps.

6.1 Attack Strategies

Our proposed exploitation is conceptually inspired by the approach of Janus vulnerability (CVE-2017-13156) [32]. Unlike that Janus exploit, which requires extensive analysis of the APK structure, our attack does not rely on parsing the signature content but requires only that the modified app be repackaged and signed with the compromised key.

To realize malicious manipulation in the repacked apps, we propose two strategies for modifying the original app packages:

(1) **Creating permission conflict**. Because many affected apps are installed as privileged applications on OEM devices, altering the permission declaration in these apps would result in permanent denial of service of the victim device. We depict this process in Figure 4 (a). Specifically, the attacker can reverse engineer a privileged app signed with the compromised certificate, and in the forged version intentionally requests more permissions than what the benign app has been explicitly granted in the allowlist[11]. By doing this, the Android OS detects an inconsistency in handling privileged-level permissions on the next boot, and as a result, will keep crashing permanently [18].

(2) **Injecting malicious DEX**. The Android OS allows developers to insert and execute pre-compiled DEX files into the apps [15]. This mechanism supports headless plugins as dynamically loaded components in Android apps [5, 37] and also enables the creation of the Janus vulnerability exploitation [32]. We, on behalf of an attacker, can similarly inject a malicious DEX payload into the original apps to abuse them. Specifically, we can first implement an attack script in Java, which usually performs network sniffing, user input monitoring, or remote execution. We then compile the attack script to DEX format to obtain an attack payload, and write a "loader" method to load the DEX payload during runtime. An example of the loader method is provided in Listing 1. After that, we compile the loader method together with the payload into Smali code[12] and insert it into code executed during application initialization. Lastly, a spoofed version of the victim app can be made by re-signing the modified package using the leaked keys. Thus,

[11] Allowlists are implemented in .XML files in the frameworks/base/etc/permissions directory [38].

[12] Smali is an assembly language used by Android, which can be used to modify and reverse engineer Android apps, and allows developers to make changes to the bytecode of an app [24].

our prepared attack payload can be executed each time the spoofed app is launched on the victim device. We depict this process in Figure 4 (b) and implement a proof-of-concept based on this strategy, which will be detailed in the remainder of this section.

6.2 A Proof-of-Concept

We begin by simulating a typical hacker's approach in building a malicious app, assuming that the original app can be reverse-engineered, modified, and rebuilt. In this study, we perform a preliminary analysis of the functionality of the 278 compromised apps and use a popular reverse engineering tool named `ApkTool` [39] to assess their exploitability, i.e., whether we can rebuild the decompiled app package. For demonstration purposes, we aim to build a forged app while minimizing code modifications and avoiding, where possible, application-level integrity checks like online verification and file checksum validation. For those reasons, we exclude the apps whose core functionality depends heavily on network connectivity, and as a result, we selected the Baidu input method as the victim app.

We assume that the victim app has access to the user's keyboard input and accordingly craft an attack script to read and save users' keyboard input into a file located in the public storage (i.e., the `/sdcard` directory). We then compile the script and obtain the attack payload as a DEX file. Next, we write a loader method to load the attack payload at runtime and invoke the attack methods. We compile the loader to Smali code, and inject it into code executed whenever the victim app is launched. Lastly, we use the compromised key to sign the modified package with a higher version[13], and then we obtain the forged app.

To facilitate demonstration, we also write an auxiliary app to actively read the saved keyboard input from the storage. Thus, after the user installs the forged app (i.e., through app update), any keyboard input using the forged Baidu input method will be stealthily recorded and displayed in the auxiliary app in real time, as illustrated in Figure 4 (b). The core implementation details of this proof-of-concept, together with a demonstration video, can be found in our anonymous repository (see Section 13).

We note that this proof-of-concept only reflects the exploitability of our findings during the study period. The spoofing attack could, in principle, be applied to any compromised app, given that the attacker has sufficient reverse-engineering expertise and knowledge of the target app. However, the developer of the victim app may strengthen its security mechanism to proactively prevent possible attacks in future releases.

7 Broader Impact beyond the Mobile Platforms

A compromised keystore named "`carlife.keystore`" in a repository "`Carlife-Android-Vehicle`" drew our attention when we sorted our findings. Although we did not find an app matching the signing key, it still raised serious concern because its filename appeared to reference a popular automotive platform owned by Baidu. The same keystore can also be found in our findings with filename "`debug.keystore`" in both the same project and another repository named "`DuerOS-Launcher`".

[13]We build the forged app with version `v11.6.12.17` to upgrade the benign one `v.11.6.12.5`

We then performed a dedicated investigation of the found keystore to assess the potential risk and impact. Specifically, we searched for candidate apps designed either for the Android mobile platform or for the vehicle platform. We found a publicly available app named "CarLife+" that can only be downloaded through the Baidu CarLife official website. By extracting the APK's signature, we managed to match the SHA-256 digest of the certificate with the private key included in the compromised keystore. We also learned that Baidu CarLife is one of the most widely used automotive head-unit platforms with the largest user base in China, and DuerOS is the vehicle-to-everything (V2X) OS that works with Baidu CarLife. According to these findings, we conclude that the signing key for CarLife has been leaked and the impact of the signature leakage incident has extended to smart vehicle platforms. An attacker can take advantage of this compromised keystore to launch a spoofing attack by replacing the original CarLife app on the victim's phone, and therefore the fake CarLife app can collude with other malicious apps on the device to collect private user data and monitor driving behavior. Even without a colluding app, the fake CarLife app can still interrupt the normal operation of the automotive head unit in the vehicle and conduct a DoS attack, such as displaying pop-up advertisements or blank screens, as conceptually illustrated in Figure 5. Similar attack approaches can pose even more severe safety risks in future generations of smart vehicles when autonomous driving becomes more prevalent.

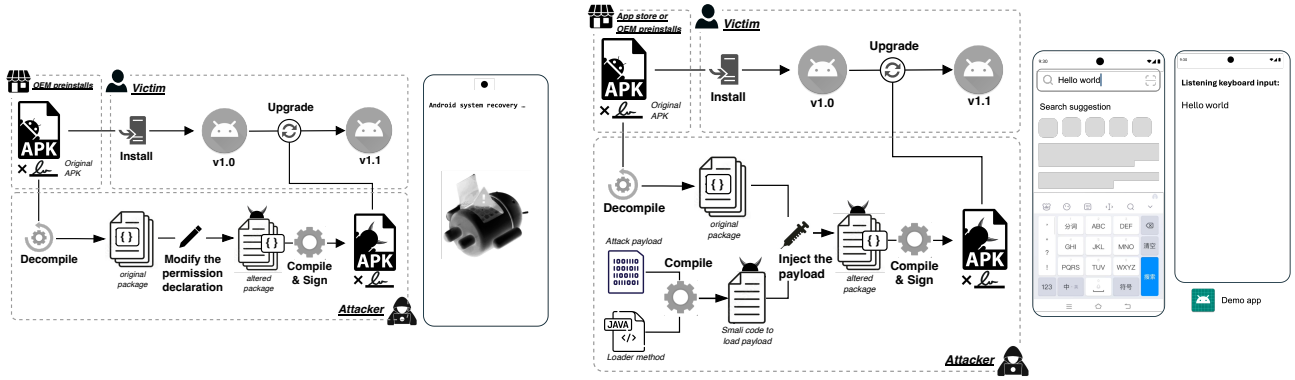
We note that the potential impact of the signing-key leakage issues in the Android ecosystem might be more severe than that on other platforms because Android OS is widely deployed not only on mobile devices but also in vehicles, IoT devices, home appliances, and other smart-home systems. The affected population may be unexpectedly large, and the mitigation could be an extremely costly and lengthy process. For example, CarLife has been installed in more than 1,100 vehicle models produced by 99 manufacturers in the Chinese market, including the global best-selling ones like Toyota, Ford, Hyundai, Audi, BMW, Volkswagen, and Mercedes, dominating the automotive head unit market share in China [3].

8 Ethical Disclosure

We responsibly disclosed our confirmed findings to the relevant parties and withheld case-specific details for at least 90 days prior to publication. For affected third-party apps, we manually identified developer contact channels and reported the observed signing-key exposure and associated risks. Remediation progress remained uneven at the time of writing. While some developers acknowledged the reports, only one third-party developer had completed key rotation among the cases listed in Table 1.

Unlike third-party applications, which may require lengthy signing key rotation processes, system apps embedded in firmware images can often be addressed more efficiently through firmware or OS updates. Although only a subset of vendors formally acknowledged the issues, we confirmed that the affected system apps have since been addressed through firmware updates.

We also observed that the repositories involved in these confirmed cases were no longer publicly accessible, which reduces immediate re-exposure risk, although it does not necessarily imply full remediation of the affected signing identities.



(a) (b)

Figure 4: The process of (a) creating permission conflict and (b) injecting malicious DEX.

```
public static void loadMemoryDex(String payload, String
     className, String methodName, String params) {
  try {
    // Initiate an InMemoryDexClassLoader object using the
        attack payload
    Object dexClassLoaderObj = Class.forName("dalvik.system.
        InMemoryDexClassLoader")
      .getDeclaredConstructor(ByteBuffer.class, ClassLoader.
          class)
      .newInstance(ByteBuffer.wrap(Base64.getDecoder().decode(
          payload)), Object.class.getClassLoader());
    // Initiate the findClass method
    Method findClassMethod = Class.forName("dalvik.system.
        BaseDexClassLoader")
      .getDeclaredMethod("findClass", String.class);
    findClassMethod.setAccessible(true);
    // Load the attack method defined in the payload
    Class<?> dexTaskClass = (Class<?>) findClassMethod.invoke(
        dexClassLoaderObj, className);
    // Execute the attack by direct invocation
    dexTaskClass.getDeclaredMethod(methodName, String.class,
        String.class).invoke(null, params);
  } catch (Exception e) {
    e.printStackTrace();
  }
}
```

Listing 1: Code for DEX injection used in our proof-of-concept exploit

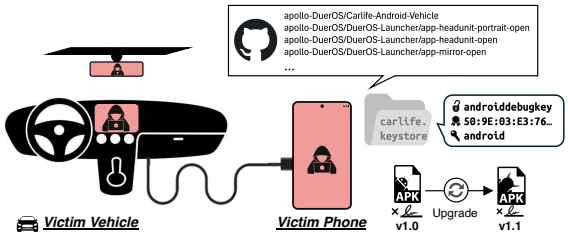


Figure 5: An illustrated attack to hijack the automotive head unit in the vehicle through the victim's smartphone after the private key being compromised

9 Recommendations

Our findings point to recurring weaknesses in the handling of Android signing credentials across both third-party development and OEM deployment settings. Based on these observations, we offer several practical recommendations to reduce the risk of signing-key exposure and limit its downstream consequences.

Strengthen Credential Management. Developers should treat app-signing credentials as long-lived security-critical assets rather than ordinary build artifacts. Keystore passwords should never be stored in plaintext within project files or other repository-accessible locations, and weak passwords should be avoided. Where supported by the signing toolchain, keystore and key credentials should be managed separately and protected through independent access controls. Signing credentials should also be stored in environments protected by multi-factor authentication and least-privilege access controls. All these policies should be reinforced through automated release-engineering controls. In particular, developers should integrate lightweight static-analysis checks into pre-commit hooks and CI/CD pipelines to detect signing artifacts (e.g., `.keystore` and `.jks` files), plaintext credentials, and insecure signing configurations before code is published or release artifacts are generated. Future work may investigate whether AI-assisted auditing can complement deterministic checks by identifying insecure signing configurations that require contextual or cross-file analysis.

Delegate App Signing to Trusted Services and Support Key Rotation. To reduce the risk of key exposure, developers should consider delegating signing operations to trusted managed services, e.g., Google Play App Signing [11]. Such services isolate long-lived app-signing keys from local development environments and may support controlled upload-key reset or signing-key upgrade when developer-side credentials are compromised. This approach is particularly beneficial for teams without mature internal processes for secret management and secure release engineering. Future work should also explore rotation-aware signing infrastructures that

reduce developers' direct and persistent access to long-lived private keys. Because Android relies on signing-identity continuity to authenticate application updates, routinely replacing app-signing certificates with short-lived certificates could disrupt update compatibility across app versions and distribution channels. A more practical direction would be to combine protected long-term signing identities with short-lived signing authorization, ephemeral CI credentials, isolated remote-signing sessions, and auditable key-rotation mechanisms.

Enforce Stricter OEM App-Signing Discipline. Device manufacturers (i.e., OEMs) should never use publicly available AOSP test or platform keys to sign production system components. OEMs should instead sign OS images and security-critical components with OEM-specific keys in isolated and access-controlled environments. They should also avoid signing third-party apps with system or platform keys, or otherwise granting such apps system-level privileges unless strictly necessary. These practices unnecessarily expand the impact of a signing-key compromise and may turn vulnerabilities in third-party apps into vectors for privilege escalation.

10 Threats to Validity

Our methodology may have a few internal constraints arising from its design and scope. Our work is also subject to several external limitations outside our control.

10.1 Internal Limitations

Repository and File-Format Scope. Our repository-side analysis is centered on Android projects and focuses on the default Java keystore formats commonly used in Android release workflows, namely `.jks` and `.keystore`. This design supports a consistent and reproducible extraction pipeline, but it may miss signing materials stored in non-Android repositories or in other formats such as `.pem` and `.p12`. As a result, our measurements may understate the broader landscape of signing-key exposure.

Password Leakage Criteria. We conservatively classify a key as compromised only when the corresponding plaintext password appears directly in the project configuration or can be resolved through an accessible file path referenced by the repository. Our study does not consider more complex password-management practices, encrypted storage, or organization-specific signing workflows. In addition, although some exposed passwords appear weak, we do not attempt password cracking for ethical and legal reasons. These choices likely cause our analysis to underestimate the scale of weak credentials in the real world.

Repository Coverage. Our analysis focuses on public GitHub repositories because of their scale and relevance to modern software development [25]. However, Android projects may also be hosted on other platforms, such as GitLab, Bitbucket, and Gitee. Leakage patterns on those platforms may differ, which limits the generalizability of our measurements to the broader repository ecosystem.

10.2 External Limitations

App Source Selection. Our third-party app collection relies on public app-store rankings and retrieves APK files through APKPure, primarily to support a lightweight and reproducible pipeline for obtaining the latest app versions. We acknowledge that academic app archives such as AndroZoo [1] provide a valuable large-scale resource by mirroring Android apps from multiple sources, and could also serve as an alternative source of APKs for similar studies.

After verifying the affected third-party apps identified in this work, we found that 24 out of the 26 compromised third-party apps can also be located in AndroZoo. This suggests that our findings are unlikely to be driven solely by the choice of APKPure as the download source. Nonetheless, the exact availability and version alignment of APKs may vary across providers, which may affect the completeness and reproducibility of app collection in future replications.

Search and Retrieval Constraints. Systematic large-scale collection from GitHub is affected by evolving search restrictions and limitations on external tool access. Although we adopted practical workarounds, these constraints may reduce the completeness and reproducibility of data collection.

Temporal Limitation. Our dataset is derived from snapshots collected during the study period rather than from a complete historical archive of repository states. Developers may subsequently add, remove, or rotate signing keys. Therefore, the reported landscape should be interpreted as a conservative snapshot and may not capture all historical instances of signing-key exposure or newly emerging leakage patterns.

Exploitation Complexity. While we demonstrate a proof-of-concept attack using a compromised key, the practical impact of such attacks depends on various factors, including the app's user base, the attacker's reverse engineering capabilities, and the feasibility of crafting an attack payload (e.g., triggering privileged operations and/or exfiltrating sensitive data). The absence of a general-purpose automated exploitation pipeline limits the real-world applicability of exploitation at scale.

Ethical Boundaries. We intentionally refrain from password cracking to ensure our methodology aligns with ethical norms. However, this may cause us to miss additional exploitable cases.

11 Related Work

Android App Signing and Developer Practices. The security of Android's app signing mechanism has been the subject of multiple studies, although there is still no comprehensive large-scale measurement of the security landscape through the lens of developer key leakage. Early research by Zhou and Jiang [42] explored malicious Android applications, highlighting that attackers could exploit poorly managed signing keys to deploy malware under legitimate identities. Similarly, studies by Barrera *et al.* [4] and Felt *et al.* [7] evaluated permission models and developer practices, noting that security vulnerabilities often stem from developers' insufficient security knowledge and poor credential management.

Relevant work, such as Egele *et al.* [6], assessed the Android ecosystem's robustness by analyzing apps' cryptographic practices, concluding that many apps suffered from severe cryptographic misuse or poor key management practices. Wang *et al.* [40] comprehensively investigated the security mechanism of early versions of

Android app signing schemes and proposed a taxonomy covering different types of app signing issues. The authors of [40] also indicate that some amateur Android developers use publicly known keys to sign their apps. Once such keys are leaked, attackers can craft malicious apps bearing authentic signatures. More recently, Wei *et al.* [41] performed a systematic assessment of secret leakage within the app packages and identified over 3,700 exploitable app secrets, underscoring the worrisome security practices in the Android developer community. Shi *et al.* [34] studied credential leakage of mini-apps on their hosting super-app platforms, uncovering over 84,000 credential leaks spanning 54,000 mini-apps.

To the best of our knowledge, this is the first study that examines the Android signing key mishandling at scale and systematically investigates the impact in the broader ecosystem.

Mining Software Repositories for Credential Leakage and Mobile Security. There is a growing body of work that applies mining software repositories techniques to detect credential and key leakage. Meli *et al.* [26] conducted a large-scale longitudinal analysis of billions of files across GitHub and searched for private RSA key leakage from them, showing that hard-coded passwords and API keys were among the most prevalent causes of exposure. Sinha *et al.* [35] proposed a hybrid detection approach combining keyword search, pattern matching, heuristic filtering, and code slicing to identify API keys in large GitHub repositories. Shi *et al.* [33] studied Android mobile payment credential leakage by mining public Git repositories and demonstrated how attackers can exploit them to forge package signatures and hijack payment flows in third-party cashier systems. Feng *et al.* [8] proposed PassFinder, a deep-learning-based system to explore password leakage in GitHub repositories. It leverages deep neural networks to unveil the intrinsic characteristics of textual passwords and comprehend the code semantics that use these passwords for authentication purposes.

Prior research on repository mining has demonstrated mature techniques for detecting credential leakage by inspecting the code base. Our research builds on the idea of mining software repositories and complements existing literature with a focus on app signing as part of release engineering in the Android app-development lifecycle.

12 Conclusion

In this work, we investigate the security landscape of app signing in the Android ecosystem with a particular focus on developer-side key management. We conduct a large-scale measurement covering 11,526 keys collected from public repositories and 2,225 developer signatures extracted from over 4,000 apps. By recovering passwords exposed by repository owners, our analysis finds 26 compromised certificates threatening the security of 278 apps and affecting over 10 billion users. We also implement a proof-of-concept attack on a real device to demonstrate the practical exploitability of the compromised keys. Our further analysis reveals that the impact of signing key leakage already extends beyond mobile devices and threatens the security of vehicles. Our findings highlight the need for developers and OEMs to adopt rigorous security practices throughout the development lifecycle and for the industry to deploy stronger key-management solutions.

13 Data Availability Statement

Our artifacts are made available for anonymous review at https://doi.org/10.5281/zenodo.19336489. We have also uploaded a demonstration video of the proof-of-concept exploit to the repository.

References

[1] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: Collecting Millions of Android Apps for the Research Community. In *Proceedings of the 13th International Conference on Mining Software Repositories* (Austin, Texas) *(MSR '16)*. ACM, New York, NY, USA, 468–471. doi:10.1145/2901739.2903508

[2] AppBrain. 2024. AppBrain Statistics - Top manufacturers. (accessed 20 March 2026). https://www.appbrain.com/stats/top-manufacturers

[3] Baidu. 2025. Baidu CarLife+. (in Chinese, accessed 5 March 2026). https://carlife.baidu.com/carlife/morecars

[4] David Barrera, H Güneş Kayacik, Paul C Van Oorschot, and Anil Somayaji. 2010. A methodology for empirical analysis of permission-based security models and its application to android. In *Proceedings of the 17th ACM conference on Computer and communications security*. 73–84. doi:10.1145/1866307.1866317

[5] Kerry Bisset. 2023. Exploring the Frontier of Android Plugin Architecture: For the Curious and the Innovative. (accessed 11 March 2026). https://medium.com/%40kerry.bisset/exploring-the-frontier-of-android-plugin-architecture-for-the-curious-and-the-innovative-3cb903a4decd

[6] Manuel Egele, David Brumley, Yanick Fratantonio, and Christopher Kruegel. 2013. An empirical study of cryptographic misuse in android applications. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 73–84. doi:10.1145/2508859.2516693

[7] Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. 2011. Android permissions demystified. In *Proceedings of the 18th ACM conference on Computer and communications security*. 627–638. doi:10.1145/2046707.2046779

[8] Runhan Feng, Ziyang Yan, Shiyan Peng, and Yuanyuan Zhang. 2022. Automated detection of password leakage from public github repositories. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. 175–186. doi:10.1145/3510003.3510150

[9] Google. 2023. Privacy changes in Android 10. (accessed 9 March 2026). https://developer.android.com/about/versions/10/privacy/changes

[10] Google. 2023. Privacy in Android 11. (accessed 9 March 2026). https://developer.android.com/about/versions/11/privacy

[11] Google. 2023. Sign your app. (accessed 3 March 2026). https://developer.android.com/studio/publish/app-signing

[12] Google. 2024. Application Signing. (accessed 3 March 2026). https://source.android.com/docs/security/features/apksigning

[13] Google. 2024. System Security Best Practices. (accessed 3 March 2026). https://source.android.com/docs/security/best-practices/system

[14] Google. 2025. Configure your build. (accessed 3 March 2026). https://developer.android.com/build#top-level

[15] Google. 2025. InMemoryDexClassLoader. (accessed 11 March 2026). https://developer.android.com/reference/kotlin/dalvik/system/InMemoryDexClassLoader

[16] Google. 2025. <permission>. (accessed 3 March 2026). https://developer.android.com/guide/topics/manifest/permission-element

[17] Google. 2025. Permissions on Android. (accessed 3 March 2026). https://developer.android.com/guide/topics/permissions/overview

[18] Google. 2025. Privileged permission allowlist. (accessed 13 February 2026). https://source.android.com/docs/core/permissions/perms-allowlist

[19] Google. 2025. Sign builds for release. (accessed 3 March 2026). https://source.android.com/docs/core/ota/sign_builds

[20] Google. 2026. Google Play Protect. (accessed 11 March 2026). https://developers.google.com/android/play-protect

[21] Google Groups. 2025. What's the difference of /system/app/ and /system/priv-app in Kitkat? (accessed 3 March 2026). https://groups.google.com/g/android-platform/c/Vsa4IsJcOPE

[22] Umar Hussain. 2018. Storing Android Signing Config Credentials: Secure and Platform Independent. (accessed 11 March 2026). https://medium.com/%40umar.hussain/storing-android-signing-config-credentials-secure-and-platform-independent-c593464f927c

[23] IBM. 2024. What is Log4Shell? (accessed 11 March 2026). https://www.ibm.com/think/topics/log4shell

[24] Rahul Kumar. 2023. An Introduction to Smali. (accessed 5 March 2026). https://payatu.com/blog/an-introduction-to-smali/

[25] Charlie Lambropoulos. 2022. GitHub, GitLab, Bitbucket. Which code repository do you use? (accessed 11 March 2026). https://www.scrumlaunch.com/blog/GitHub-GitLab-Bitbucket

[26] Michael Meli, Matthew R McNiece, and Bradley Reaves. 2019. How bad can it git? characterizing secret leakage in public github repositories. In *Proceedings of*

the Network and Distributed Systems Security (NDSS) Symposium. doi:10.14722/ndss.2019.23418

[27] Lily Hay Newman. 2022. Android Phone Makers' Encryption Keys Stolen and Used in Malware. (accessed 11 March 2026). https://www.wired.com/story/android-platform-certificates-malware/

[28] NIST. 2013. CVE-2013-4787. (accessed 11 March 2026). https://nvd.nist.gov/vuln/detail/CVE-2013-4787

[29] NIST. 2017. CVE-2017-13156. (accessed 11 March 2026). https://nvd.nist.gov/vuln/detail/cve-2017-13156

[30] NIST. 2021. CVE-2021-44228. (accessed 11 March 2026). https://nvd.nist.gov/vuln/detail/cve-2021-44228

[31] Raul Roberts. 2024. A (Partial) History of Software Supply Chain Attacks. (accessed 11 March 2026). https://www.reversinglabs.com/blog/a-partial-history-of-software-supply-chain-attacks

[32] Sanatsu. 2021. Exploiting Apps vulnerable to Janus (CVE-2017–13156). (accessed 11 March 2026). https://medium.com/mobis3c/exploiting-apps-vulnerable-to-janus-cve-2017-13156-8d52c983b4e0

[33] Shangcheng Shi, Xianbo Wang, Kyle Zeng, Ronghai Yang, and Wing Cheong Lau. 2021. An empirical study on mobile payment credential leaks and their exploits. In *International Conference on Security and Privacy in Communication Systems*. Springer, 79–98. doi:10.1007/978-3-030-90022-9_5

[34] Yizhe Shi, Guangliang Yang, Zhemin Yang, Yifan Yang, Min Yang, Kangwei Zhong, and Xiaohan Zhang. 2025. The Skeleton Keys: A Large Scale Analysis of Credential Leakage in Mini-apps. In *Proceedings of the Network and Distributed Systems Security (NDSS) Symposium*. doi:10.14722/ndss.2025.230273

[35] Vibha Singhal Sinha, Diptikalyan Saha, Pankaj Dhoolia, Rohan Padhye, and Senthil Mani. 2015. Detecting and mitigating secret-key leaks in source code repositories. In *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*. IEEE, 396–400. doi:10.1109/MSR.2015.48

[36] Sreelakshmi S Dilip. 2025. Android Automotive: Understanding app permissions and privileged applications. (accessed 3 March 2026). https://medium.com/{@}sreelakshmis.dilip/android-automotive-understanding-app-permissions-and-privileged-applications-c00b83df11d6

[37] StackOverflow. 2010. Is it possible to inject code in an android application? (accessed 11 March 2026). https://stackoverflow.com/questions/4260231/is-it-possible-to-inject-code-in-an-android-application

[38] StackOverflow. 2024. Android O (api 26): root-app not recognized as "priviledged". (accessed 20 March 2026). https://stackoverflow.com/a/52162905

[39] Connor Tumbleson. 2025. Apktool: A tool for reverse engineering Android apk files. (accessed 11 March 2026). https://apktool.org/

[40] Haoyu Wang, Hongxuan Liu, Xusheng Xiao, Guozhu Meng, and Yao Guo. 2019. Characterizing android app signing issues. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 280–292. doi:10.1109/ASE.2019.00035

[41] Lili Wei, Heqing Huang, Shing-Chi Cheung, and Kevin Li. 2025. How far are app secrets from being stolen? a case study on android. *Empirical Software Engineering* 30, 3 (2025), 90. doi:10.1007/s10664-024-10607-9

[42] Yajin Zhou and Xuxian Jiang. 2012. Dissecting android malware: Characterization and evolution. In *2012 IEEE symposium on security and privacy*. IEEE, 95–109. doi:10.1109/SP.2012.16

Received 2026-03-26; accepted 2026-06-18